Member-only story

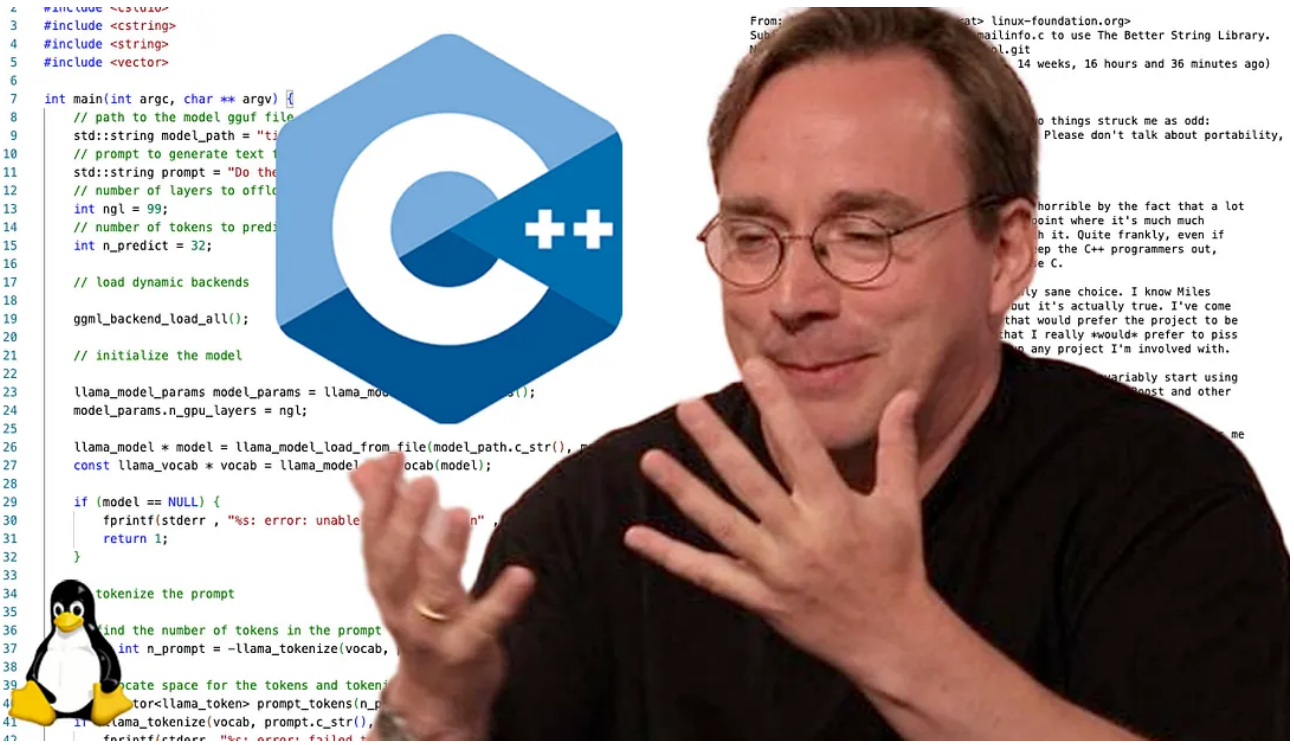
Linus Torvalds' Critique of C++: A Comprehensive Review

11 min read · Feb 14, 2025

Jan Kammerath Follow

Listen More

Linus Torvalds, the inventor *(and beloved dictator)* of Linux, has always been quite harsh about C++ and why he rejects it for Linux kernel development. He's not just been very vocal about it, but also brought up some arguments against the use of C++ that are worth reviewing in detail.



Why the hate on C++? Let's focus on the arguments against it.

C and C++ are almost the same, except they aren't. C++ is often considered the object oriented "version" of C. The successor to it, if you will. But C++ is rather an extension to C. Bringing object orientation, constructors, destructors, templates, exception handling, namespaces and operator overloading. All these extensions come with their very own challenges and paradigms. Unsurprisingly, Linus' technical arguments against C++ all have their roots in these extensions.

Linus' arguments against C++

The main arguments that Linus brought forward against C++ can be found in the kernel mailing list under "Compiling C++ kernel module + Makefile" and the respective responses that date back as far as 2004. Let's wipe the personal disputes off these messages and focus on the arguments.

Exception handling in C++

> *"the whole C++ exception handling thing is fundamentally broken. It's _especially_ broken for kernels."*

C relies on return values to indicate errors while C++ encourages the use of exceptions. Exceptions can throw in any part of the code and hence make the code, that throws them, unpredictable to some extent. They are non-deterministic and give C++ an entirely different paradigm for error handling. Error handling is a very basic necessity for programming and the different approach of C++ essentially makes it an entirely different programming language.

That unpredictability and the variability of how exceptions are implemented makes them really difficult to debug compared to C's simple approach with return values. Introducing exceptions to the Linux kernel would result in the kernel having different approaches to error handling, at least during several years of potentially migrating everything to C++.

While all this isn't really a problem when writing a paint application for GNOME, it is a real risk and threat to the Linux kernel with its 30 million lines of code. Introducting exception handling to the Linux kernel will inevitably lead to it becoming more unstable. Although the testing and approval of kernel changes is very strict, it is impossible to totally avoid all errors or mistakes.

Memory management in C++ compilers

> *“any compiler or language that likes to hide things like memory allocations behind your back just isn’t a good choice for a kernel”*

Exception handling does bring overhead that is hidden away by the compiler. This argument about “hidden memory mangement” points at RAII *(Resource Acquisition Is Initialization)* or the automatic memory management through destructors in C++.

The Linux kernel has very fine-grained and fine-tuned memory managment. Many performance optimizations have pushed the limits of compilers. In a situation where the kernel development team is already facing challenges with compilers, it is risky to introduce such automated compiler features.

Not just will it inevitably lead to lower performance of modules using RAII, it will also introduce a new dependency. The Linux kernel would be more dependent on the compiler. Although this is an extremely small dependency and very minimal, it becomes a larger problem with a 30 million line code base running on billions of devices across the globe.

Object orientation in C++ vs. C

> *“you can write object-oriented code (useful for filesystems etc) in C, _without_ the crap that is C++”*

Object oriented programming (OOP) is the main argument for using C++ in favor of plain C. Linus’ argument here is that the weight and risk introduced by C++ is not worth it. He is making the point that you can do basic OOP in C as well. This is possible by using structs *(structures)* that mimic C++ classes. The below code outlines a very simple example on how object orientation can be achieved in plain C.

```c
#include <stdio.h>
#include <stdlib.h>

// Define the "class" using a struct
typedef struct {
    int value;
    // Function pointer for a method
    void (*increment)(struct Person *self);
```

```
} Person;

// Method implementation for increment
void increment_person(Person *self) {
    self->value++;
}

// Constructor-like function to initialize a Person
Person* person_new(int initial_value) {
    Person *p = (Person*)malloc(sizeof(Person));
    p->value = initial_value;
    p->increment = increment_person;
    return p;
}

// Destructor-like function
void person_free(Person *p) {
    free(p);
}

int main() {
    // Creating an instance of Person
    Person *p = person_new(5);

    // Using the method
    printf("Initial value: %d\n", p->value);
    p->increment(p);  // Incrementing the value
    printf("After increment: %d\n", p->value);

    // Freeing the allocated memory
    person_free(p);
    return 0;
}
```

OOP is not a technical requirement for the Linux kernel to function. It is merely a different paradigm, or concept. The purpose of introducing OOP to the Linux kernel would be to improve modularity, encapsulation and code reusability, thus simplify maintenance and development. These are all advantages that fall into the category of developer ergonomics.

The Linux kernel is widely adopted on billions of devices across the globe because of its performance and stability. These advantages come at the cost of developer ergonomics and time to market. Any introduction of better developer ergonomics will inevitably lead to a weakening of both the kernel's performance and stability. Performance, stability and ergonomics are always a trade off.

The Linux kernel development largely neglects ergonomics for its developers to achieve the maximum possible stability and performance. That is the philosophy of the kernel. Introducting OOP is hence not just a risk and threat, but a total change of philosophy. Something that Linux users, especially in mission critical environments, will neither welcome nor tolerate. Again, a valid point made by Linus.

Stability of libraries and dependencies in C++

> *"infinite amounts of pain when they don't work (and anybody who tells me that STL and especially Boost are stable and portable is just so full of BS that it's not even funny)"*

Boost and STL can be considered "stable" in the context of user level or userspace application development. When it comes to kernel development and the requirements that the Linux kernel has, the term "stable" has a far stricter meaning.

The request to use C++ with the Standard Template Library (STL) or the Boost Libraries is brought up with the intention to improve developer ergonomics again. Introducting such dependencies comes with the same risks and threats to the kernel as RAII. They, again, would be introduced at the cost of performance and stability.

Besides performance and stability, there's the challenge of ownership. Every dependency introduced to the kernel, transfers responsibility to the maintainer of that dependency. With that transfer of responsibility also comes a transfer of ownership which bears security risks. Risks such as CVE-2024–3094, a backdoor in the liblzma library, introduced by a mysterious user named "Jia Tan".

Inefficient and bloated abstraction

> *"inefficient abstracted programming models where two years down the road you notice that some abstraction wasn't very efficient, but now all your code depends on all the nice object models around it, and you cannot fix it without rewriting your app."*

These claims have less to do with C++ specifically, but rather focus on OOP and the concept of inheritance. Design patterns in object oriented programming aim

to provide blueprint solutions to reoccuring problems. They are useful and anyone who gets into OOP should take note of them.

"Dare you enter my abstract factory?" (Source: /r/ProgrammerHumor on Reddit)

The kernel developers are very capable software engineers and programmers. They know how to apply OOP, when and where. Yet, they know that OOP doesn't come without downsides. Besides technical considerations, such as memory overhead and larger binaries, there are logical pitfalls that even experienced programmers aren't immune to.

Linus' argument is that a wrongfully applied class hierarchy and structure will inevitably lead to unmaintainable code. Creating bloated class hierarchies is not unusual with OOP. Although the kernel developers would certainly not fall for that on a large scale, they might on a smaller scale. But in a much larger codebase which is similarly dangerous.

Regardless of how good your code technically is, if you logically mess up your class hierarchy, structure and design, you can get into the dead end of maintainability. So much so that the logical class structure needs a complete overhaul. It is not unusual for OOP applications to have to undergo complete restructuring of the application architecture after only a couple of years. There is a chance that the Linux kernel might slip down that slope. Is this risk really worth taking?

You'll end up with plain C anyway

> *"In other words, the only way to do good, efficient, and system-level and portable C++ ends up to limit yourself to all the things that are basically available in C."*

None of the aforementioned features are required or mandated in C++. You can perfectly write C++ code the way you write code in plain C. Linus' argument here is: Why bother with C++ then? If you write C++ code in C-style, the C++ compiler becomes an end in itself.

> *"And limiting your project to C means that people don't screw that up, and also means that you get a lot of programmers that do actually understand low-level issues and don't screw things up with any idiotic "object model" crap."*

This second argument in the last paragraph of his mailing list post from September 6th, 2007 is around developer focus when sticking to C. The core message of this statement is that C will attract kernel developers who are focused

on hardware and solving the actual challenges without getting lost in the beauty or confusion of OOP.

Developers are humans after all and this argument outlines the importance of people in programming. It shows that Linus' as the BDFL (Benevolent Dictator for Life) of the Linux world needs to consider people management as much as any other leader. He is the godfather of probably the largest community of developers on this planet and should keep it that way. He, to some extent, has to deal with and manage the expectations, hopes and dreams of over 20 million developers worldwide.

What's next, Rust, Go or even native Java?!

> *"C is, in the end, a very simple language. It's one of the reasons I enjoy C and why a lot of C programmers enjoy C, even if the other side of that picture is obviously that because it's simple it's also very easy to make mistakes, and Rust is not." — Keynote: Linus Torvalds in Conversion with Dirk Hohndel*

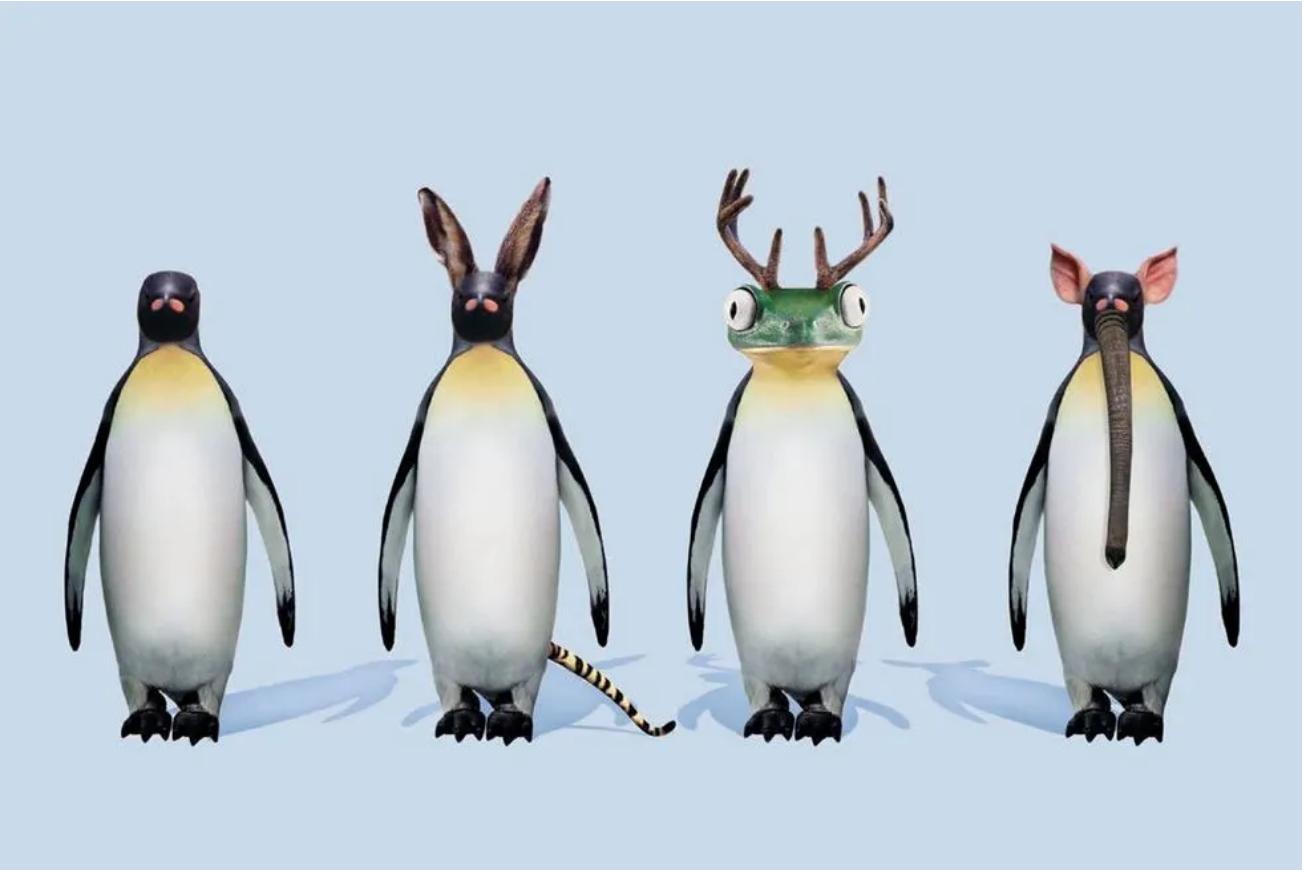
Unsurprisingly, and just added as an honorable mention, there is already a debate of using Rust in the Linux kernel. What's next; Go, C# or even Java compiled into a native binary with GraalVM?! The intention of the developers asking these questions are understandable. It's the same intention as with C++, and it is not ill intended.

Linus has to draw a line somewhere. Linux is 99% plain C and the remaining is pure Assembly. The architecture of the kernel is the cleanest architecture of any operating system kernel in human history. The challenges of the kernel aren't necessarily only programming challenges, they're technical challenges involving interrupt handling, context switches and other low-level CPU operations.

The kernel needs to provide device drivers, file systems, the outstanding network stack that Linux has, its memory management, process scheduling and so much more. All that has a high technical complexity. Adding any complexity that does not directly relate to these challenges and further does not directly result in any benefits for the users, is highly questionable. The demand of Rust code in the Linux kernel adds way more complexity than C++ would, so the answer is understandably "No".

Is the rejection of C++ justified?

Yes and No as it depends. The arguments brought up by Linus would be totally laughable and ridiculous in the context of user level applications such as a paint application. In the context of kernel development, they are really important and valid arguments as the they outline the criticality of kernel development.



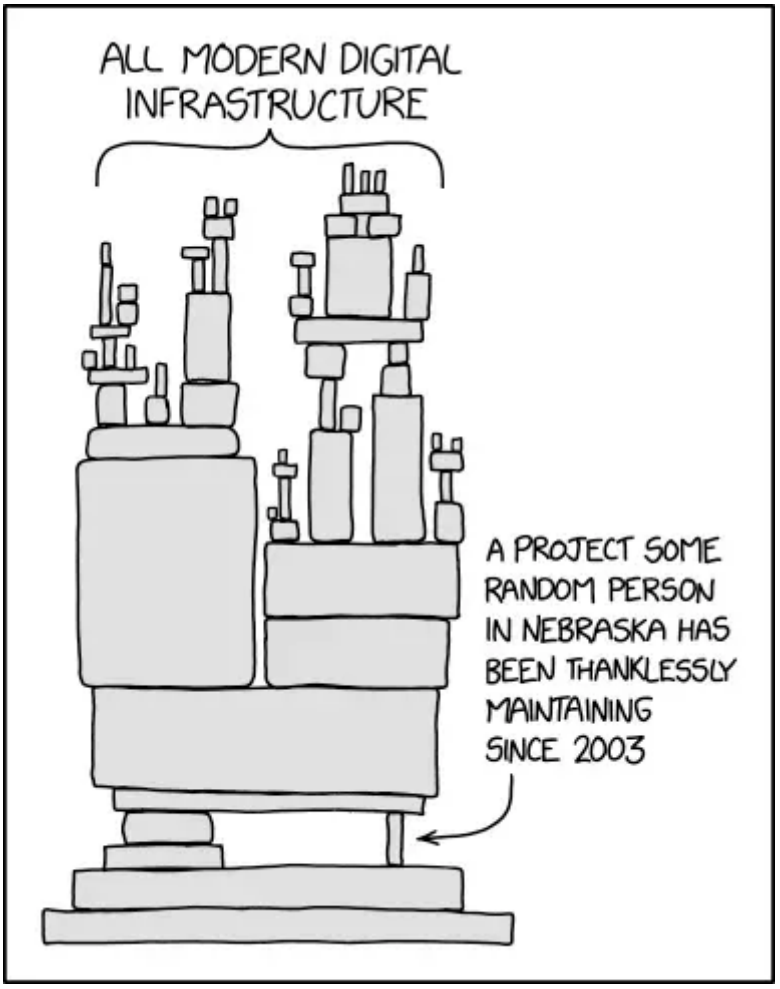
"Open operating systems do not just have advantages"; Microsoft Ad, 2004.

The success of Linux roots in its performance and stability which it achieved by having a very optimized 99% pure C codebase with only 1% Assembly. The introduction of C++, exception handling, object orientation or any other untested, fancy new technology will inevatibly result in a negative impact on both. While it sounds ridiculous to call C++ a fancy new technology or even untested, it clearly is for the Linux kernel.

What we can learn from Linus' rejection of C++

The Linux kernel is a globally unique marvel of software engineering. Over 30 million lines of code running on billions of machines. It's performance and stability are unrivaled. A direct result of the kernel developers' focus and dedication. The still ongoing debate about systemd vs. SysVinit shows how emotional, yet technically detailed, major changes to Linux are.

A “Dependency Hell” often found in Node.js applications is the exact opposite of the carefully crafted 30 million lines of code that make up the Linux kernel. Our key takeaway from the Linux kernel debate around C++ can be that we should make very informed and rational decisions about dependencies, including the consideration of long term effects.



Programmers need to be more careful with dependencies

Further, Linus’ approach to C++ or even Rust kernel development shows how important “the right tool for the job” is. Writing everything in plain C has standardized the Linux kernel, yet it still faces challenges that the kernel developers are working hard on every day. When your coworker comes along tomorrow and wants to implement a feature of your Go application in Bython, you could potentially consider Linus’ approach.

Closing argument

Next time you’re being shoved into an MRI scanner at a hospital, ask yourself if you want that thing to be able to throw exception. While it’s certainly funny when

“AI Smart Fridges” become self conscious and eat away your snacks, having the MRI go its own way can be lethal.

Do you want an MRI scanner’s kernel to throw exceptions?

The Linux kernel is in everything today: all the smart things including computers, phones, servers, network equipment, embedded systems, gaming consoles, medical devices, home appliances, wearables, automobiles, traffic lights, industrial machinery, spacecraft systems, railway control systems, ATMs, air traffic control systems, satellites, scientific instruments, medical appliances, farm machinery, space telescopes, submarines, fighter jets, and cruise missiles, only to name a few. Do you want to risk breaking them?

While Linus’ is obviously annoyed by having to answer the question about C++ in kernel development a thousand times, it is important to ask and review it. Although the answer is “No”, there is something very important we can learn from that answer: **Is it fancy or necessary? Do I do this for me or for my users? What are the long term effects of doing this?**